

Introduction

A login system shipped. Every feature worked. Sign up worked. Log in worked. Password reset worked. QA signed off. Three hours after launch, ten thousand people tried to log in at the same time, and the whole thing fell over.

Nobody skipped a feature. The spec was fully implemented. So what actually got missed?

This is one of the most common failure patterns in real systems, and it's also one of the favorite traps in system design interviews. Interviewers rarely ask "does your design work?" They ask "does your design work at scale, under load, with acceptable latency, without going down?" Those are different questions, and if you don't know how to separate them, you'll design a system that's correct on a whiteboard and broken in production.

Understanding this distinction — functional versus non-functional requirements — is one of the highest-leverage things you can learn as a software engineer. It's not abstract theory. It's the difference between a system that survives its first real traffic spike and one that doesn't.

Problem Overview

Every requirement you're ever given, whether it comes from a product manager, a spec doc, or an interviewer, answers one of two questions:

1. **What must the system do?** — This is a functional requirement. It describes a capability, a feature, a piece of behavior. "A user can log in." "A user can reset their password." "A user can search for a product." You can check these off with a simple yes or no. Either the feature works or it doesn't.
2. **How well must it do it?** — This is a non-functional requirement. It describes a constraint on quality: speed, capacity, reliability. "The response must return in under 200 milliseconds." "The system must be available 99.9% of the time." "It must handle 10,000 concurrent users."

Functional requirements tell you what to build. Non-functional requirements tell you how to build it — and they're the ones that actually decide your architecture.

The trap is that non-functional requirements are invisible on a feature checklist. Nobody puts "handles 10,000 concurrent logins" on a UI mockup. But users feel it immediately the moment it's missing.

Example

Think of a restaurant. The menu tells you what dishes the kitchen serves — that's the functional side. How fast the kitchen gets a plate to your table on a packed Friday night is a completely different question — that's the non-functional side. A kitchen that can cook everything on the menu but takes forty minutes per order during a rush is not a working restaurant, even though every dish "works."

Software has the exact same split. Take one feature: **search for a product**.

System A: 100 users total, and a 2-second response time is perfectly acceptable. A single database handles this without any special design. Done.

System B: 10 million users, and the response must come back in under 50 milliseconds. That single database now can't keep up — you need caching layers, database indexing, and probably a dedicated search engine like Elasticsearch to hit that latency target at that scale.

Same feature. Same functional requirement: "search for a product." Completely different systems, because the non-functional numbers attached to that feature were different.

Intuition

Here's how an experienced engineer actually thinks about this, before writing a single line of code.

When you're handed a requirement like "a user can log in," your first instinct might be to think about the feature itself: form validation, password hashing, session tokens. That's necessary, but it's not what determines your architecture. What determines your architecture is the unwritten context around that feature — the scale it needs to run at, the speed it needs to respond in, and the failure tolerance it's allowed to have.

"Log in a user" — one server handles that without breaking a sweat.

"Log in a user, in 200 milliseconds, for 10,000 people at once" — that's a fundamentally different engineering problem, and it's exactly why teams reach for a load balancer and multiple servers instead of a single machine.

The functional requirement is identical in both cases. The architecture is not. This is the core intuition: **non-functional requirements are the actual design constraints**. Functional requirements tell you what boxes to draw. Non-functional requirements tell you how many of those boxes you need, how they're connected, and what infrastructure sits between them.

So before you design anything, you should always be asking three questions that never show up as buttons on a screen: How many? How fast? How often is failure allowed?

Brute Force Solution

The "brute force" approach to system design is to read only the functional requirements and build the simplest thing that satisfies them.

Idea: If the requirement is "a user can log in," stand up one server, one database, write the login logic, and ship it.

Algorithm: 1. Read the feature list. 2. Build each feature with the simplest possible implementation. 3. Deploy on a single instance.

Advantages: - Fast to build. - Simple to reason about. - Zero infrastructure overhead. - Perfectly fine for genuinely low-traffic, low-stakes systems.

Disadvantages: - Completely blind to scale, latency, and availability requirements. - Works fine in testing, where load is low and controlled, and fails exactly where it matters most: in production, under real traffic. - No warning signs until the system is already down.

Time and space complexity: Not the right lens here — the failure isn't algorithmic, it's architectural. A single server login flow might be $O(1)$ per request in isolation, but complexity analysis of one request tells you nothing about what happens when 10,000 requests arrive in the same second. That's the whole point of this lesson: correctness per-request and correctness under load are different measurements.

This is exactly what happened to the login system in the opening story. It passed every functional test because functional tests check one request at a time. Nobody load-tested for 10,000 concurrent logins, because that number was never written down as a requirement.

Optimal Solution

The optimal approach is to treat non-functional requirements as first-class inputs to your design, not an afterthought you bolt on after the features are done.

Here's the process:

Step 1 — Extract the functional requirements. List every feature as a checklist item. "User can log in." "User can reset password." "User can search products." Nothing about performance yet.

Step 2 — Attach numbers to each one. For every feature, ask: how many concurrent users, how fast must it respond, and what's the acceptable downtime? These numbers are your non-functional requirements.

Step 3 — Let the numbers drive the architecture, not the other way around.

Requirement type	Example	Architecture impact
Scale	10,000 concurrent logins	Load balancer + multiple app servers
Latency	Under 50ms search response	Caching layer, indexing, or search engine
Availability	99.9% uptime	Redundancy, failover, health checks

Step 4 — Compare designs against the numbers, not against the feature list. A design "passes" only when it satisfies both the functional checklist and the non-functional numbers simultaneously.

Going back to the search example: System A (100 users, 2 seconds) and System B (10 million users, 50 milliseconds) implement the identical feature but end up as entirely different systems, because Step 2 produced entirely different numbers. That's the mechanism. The feature list is constant; the numbers are the variable that reshapes the architecture.

This is also why non-functional requirements have to be gathered up front, not discovered after launch. If "10,000 concurrent logins" isn't written down before the design phase, nobody designs for it — and you get the 3-hour crash from the opening story.

Python Code

Since this lesson is about requirements analysis rather than a single algorithm, the most useful code artifact is a small, reusable way to make non-functional requirements explicit and checkable instead of implicit and forgotten. Here's a minimal requirements model you can attach to any feature:

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class FunctionalRequirement:
    """What the system must do."""
    name: str
    description: str

@dataclass
class NonFunctionalRequirement:
    """How well the system must do it."""
```

```

max_latency_ms: Optional[int] = None
min_availability_pct: Optional[float] = None
concurrent_users: Optional[int] = None

def describe(self) -> str:
    parts = []
    if self.concurrent_users is not None:
        parts.append(f"{self.concurrent_users:,} concurrent users")
    if self.max_latency_ms is not None:
        parts.append(f"under {self.max_latency_ms}ms response")
    if self.min_availability_pct is not None:
        parts.append(f"{self.min_availability_pct}% availability")
    return ", ".join(parts) if parts else "no constraints specified"

@dataclass
class Feature:
    functional: FunctionalRequirement
    non_functional: NonFunctionalRequirement

    def needs_horizontal_scaling(self) -> bool:
        # ponytail: simple threshold heuristic, not a real capacity model
        return (self.non_functional.concurrent_users or 0) > 1000

    def needs_low_latency_infra(self) -> bool:
        return (self.non_functional.max_latency_ms or float("inf")) < 100

def summarize(feature: Feature) -> str:
    nfr = feature.non_functional.describe()
    flags = []
    if feature.needs_horizontal_scaling():
        flags.append("load balancer + multiple servers")
    if feature.needs_low_latency_infra():
        flags.append("caching / indexing / search engine")

    flags_str = f" -> requires: {'', '.join(flags)}" if flags else " -> single server is fine"
    return f"{feature.functional.name} ({nfr}){flags_str}"

```

Code Walkthrough

- `FunctionalRequirement` holds the "what" — just a name and description, no numbers. This mirrors the checklist nature of functional requirements: yes/no, present/absent.
- `NonFunctionalRequirement` holds the "how well" — latency, availability, and concurrency, each optional because not every feature needs every constraint specified.
- `describe()` turns the raw numbers into the same sentence structure the lesson uses: "how many, how fast, how often it's allowed to fail."

- `Feature` bundles both halves together, because in the optimal solution, you never evaluate one without the other.
- `needs_horizontal_scaling()` and `needs_low_latency_infra()` are deliberately simple threshold checks — they exist to demonstrate the *mechanism* (numbers drive architecture decisions), not to serve as a real capacity-planning model.
- `summarize()` produces a human-readable line showing how a single feature's non-functional numbers translate directly into infrastructure decisions — exactly the System A vs. System B comparison from the lesson.

Dry Run

Let's run both search systems from the lesson through this model.

```
system_a = Feature(
    functional=FunctionalRequirement("Search for a product", "..."),
    non_functional=NonFunctionalRequirement(
        max_latency_ms=2000,
        concurrent_users=100,
    ),
)

system_b = Feature(
    functional=FunctionalRequirement("Search for a product", "..."),
    non_functional=NonFunctionalRequirement(
        max_latency_ms=50,
        concurrent_users=10_000_000,
    ),
)

print(summarize(system_a))
print(summarize(system_b))
```

System A trace: `concurrent_users=100` → `needs_horizontal_scaling()` returns `False` (100 is not > 1000). `max_latency_ms=2000` → `needs_low_latency_infra()` returns `False` (2000 is not < 100). No flags. Output:

```
Search for a product (100 concurrent users, under 2000ms response) -> single server is fine
```

System B trace: `concurrent_users=10_000_000` → `needs_horizontal_scaling()` returns `True`. `max_latency_ms=50` → `needs_low_latency_infra()` returns `True`. Both flags trigger. Output:

```
Search for a product (10,000,000 concurrent users, under 50ms response) -> requires: load balancer
```

Identical functional requirement, identical `FunctionalRequirement` object even — and the output architecture recommendation is completely different, purely because of the non-functional numbers. That's the entire lesson compressed into one dry run.

Complexity Analysis

- **Time:** `summarize()` runs in $O(1)$ — a fixed number of comparisons regardless of the numbers involved. This is correct because we're not processing a data structure; we're evaluating fixed threshold conditions on a handful of fields.
- **Space:** $O(1)$ — each `Feature` stores a constant number of fields.
- **Why this doesn't capture the real cost:** this is worth calling out explicitly — the complexity of *this code* is trivial, but the complexity of *the system it's describing* (caching layers, load balancers, distributed search) is not. That gap is exactly the point: requirements analysis is cheap, the architecture it demands is not, which is why skipping it is so costly.

Alternative Solutions

Alternative: capacity-based thresholds instead of fixed heuristics. Instead of a hardcoded `1000` check, a more realistic system would model requests-per-second against actual server throughput benchmarks. This is more accurate but requires real load-testing data you usually don't have at design time — the simple threshold version is what you'd sketch on a whiteboard in an interview, and it's good enough to make the point.

Alternative: requirements as configuration/YAML instead of code. Some teams track non-functional requirements in a spec document or YAML file reviewed by architects before implementation starts. This decouples "requirements gathering" from "code" entirely, which is arguably more correct for a real organization — the code version here exists to make the *reasoning* executable and testable, not to replace a proper requirements doc.

Both alternatives converge on the same principle as the optimal solution: numbers must be captured explicitly, early, and attached to every feature — the format is a secondary decision.

Edge Cases

- **No non-functional requirements given at all:** Treat this as "requirements gathering is incomplete," not as "assume single server." The lesson's entire point is that an unwritten number is exactly what causes production failures — silence should be a red flag you ask about, not a default you assume.
- **Extremely low concurrency (e.g., 1 user, an internal admin tool):** A single server is genuinely correct here. Don't over-architect a tool three people will use.
- **Extremely high availability with low scale:** A low-traffic system can still need 99.99% uptime (e.g., a payment webhook receiver). Scale and availability are independent dimensions — don't assume one implies the other.
- **Conflicting requirements:** Sub-50ms latency at 10 million concurrent users with a single-region deployment may be physically impossible due to network round-trip time alone. Recognizing an infeasible requirement is itself part of the analysis.
- **Requirements that change post-launch:** A system built for 100 users that suddenly needs to support 100,000 requires re-running this entire analysis — non-functional requirements aren't a one-time document, they're a living constraint.

Common Mistakes

1. **Treating "it works" as equivalent to "it's done."** A feature passing a functional test tells you nothing about its behavior under load.
2. **Only load-testing after launch, if ever.** Non-functional requirements should be validated before the system meets real traffic, not discovered by real traffic.
3. **Assuming non-functional requirements are the infrastructure team's problem.** They determine the architecture — you can't design the system correctly without them.
4. **Writing vague non-functional requirements like "fast" or "reliable."** These aren't requirements until they're numbers: "under 200ms," "99.9% uptime." A requirement you can't measure is a requirement you can't design against.
5. **Over-engineering for numbers nobody asked for.** Building a globally distributed, auto-scaling system for an internal tool with 20 users is the same mistake in the opposite direction — non-functional requirements should match actual need, not worst-case paranoia.

Interview Questions

- "How would you scale this design if concurrent users increased 100x?"
- "What's your target latency for this endpoint, and how does that change your data storage choice?"

- "How do you handle the tradeoff between consistency and availability under this system's uptime requirement?"
- "What non-functional requirements would you ask the interviewer for before starting this design?"
- "How would your architecture differ between a system needing 99.9% vs. 99.99% availability?"

Similar Problems

- **Design a URL shortener** — a classic system design problem where read/write ratio and latency requirements (not the redirect logic itself) drive the choice between SQL and key-value storage.
- **Design a rate limiter** — directly about non-functional requirements: how many requests per second, and what happens when that number is exceeded.
- **Design a notification system** — functional requirement is simple ("send a notification"), but availability and delivery-latency requirements determine whether you need a message queue.
- **Design a chat application** — functional requirements (send/receive messages) are trivial; non-functional requirements (real-time latency, millions of concurrent connections) are what make it a hard problem.

Key Takeaways

- Every requirement answers either "what must it do" (functional) or "how well must it do it" (non-functional).
- Functional requirements are a checklist. Non-functional requirements are numbers: latency, scale, availability.
- The same feature can require radically different architectures depending only on its non-functional numbers — not on the feature itself.
- Non-functional requirements have to be gathered before the design starts, not discovered after the system is in production and failing.
- When a system fails while doing exactly what it was built to do, the real question is: what number was never written down?

Watch the Video

This lesson is easier to absorb with the visual walkthrough of System A vs. System B side by side. Watch the full explanation here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series built for developers who want to actually understand the systems they're building, not just memorize interview answers. Each lesson breaks down one core software architecture or computer science concept using plain language, real comparisons, and code you can run yourself.

Call To Action

If this lesson helped the functional vs. non-functional distinction finally click, do three things: like the video on YouTube so more developers see it, subscribe to the channel for the rest of the series, and subscribe to the Substack newsletter for deeper written breakdowns like this one. Drop a comment with a system you've seen fail this exact way — production stories are the best teaching material. And if you know an engineer prepping for system design interviews, share this with them.

Quick Review

Functional vs non-functional requirement — how do you tell them apart?

Functional = what the system does (features). Non-functional = how well it does it (latency, availability, scale).

Why did the system 'pass every check' but still fail users?

Tests verified functional correctness only; no non-functional requirement (speed, uptime) was defined or checked.

Same feature, different number — what does this scenario teach?

Two systems can implement identical functional requirements yet differ entirely in the non-functional numbers behind them.

System A vs System B: why pick different architectures for the same feature?

Differing non-functional targets (e.g. 1 server vs distributed) force different architectures despite identical functionality.

What was 'the line that was missing' in the spec?

An explicit non-functional requirement (a number like latency/availability target) — without it, architecture has no real constraint.

What actually decides the architecture: features or numbers?

The numbers — non-functional requirements (scale, latency, availability) drive architecture choices, not the feature list.

software-architecture Lesson 2: Functional vs Non-Functional Requirements (what a system must do vs how well it must do it (latency, availability, scale) -- and why non-functional reqs decide the architecture) — free Python cheat sheet